

Experiment No. 7

Aim : Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Theory :

1. Dataset Source

We will use the **Churn Modelling Dataset**:

Dataset Link:

<https://www.kaggle.com/datasets/shrutechlearn/churn-modelling>

2. Dataset Description

This dataset is used to predict whether a customer will leave a bank (churn) or not.

Features:

- RowNumber – Index (not useful)
- CustomerId – Unique ID
- Surname – Customer name
- CreditScore – Customer credit score
- Geography – Country (France, Germany, Spain)
- Gender – Male/Female
- Age – Customer age
- Tenure – Years with bank
- Balance – Account balance
- NumOfProducts – Number of products used
- HasCrCard – Credit card (0/1)
- IsActiveMember – Active status (0/1)
- EstimatedSalary – Estimated salary

Target Variable:

- Exited (0 = No, 1 = Yes)

Dataset Size:

- 10,000 rows
- 14 columns

Characteristics:

- Binary classification problem
- Contains categorical and numerical data
- Requires preprocessing

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of layers of neurons.

Each neuron computes:

$$z = \sum(w_i x_i) + b, \quad a = f(z)$$

Where:

- w_i = weights
- x_i = inputs
- b = bias
- $f(z)$ = activation function

Common activation functions:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid: $f(x) = \frac{1}{1 + e^{-x}}$

Loss function (Binary Crossentropy):

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

4. Algorithm Limitations

- Requires large data for better performance
- Can overfit easily
- Black-box model (low interpretability)
- Computationally expensive
- Sensitive to hyperparameters

5. Methodology / Workflow

Steps followed:

1. Import dataset
2. Data preprocessing
 - Remove unnecessary columns
 - Encode categorical variables (Gender, Geography)
 - Feature scaling
3. Split dataset into training and testing
4. Build ANN model using Keras

5. Compile model (optimizer, loss, metrics)
6. Train model
7. Evaluate model performance
8. Tune hyperparameters

Workflow:

Data Collection → Preprocessing → Encoding → Scaling → Train-Test Split → ANN Model → Training → Evaluation

6. Performance Analysis

Metrics used:

- Accuracy
- Confusion Matrix

Interpretation:

- High accuracy indicates good prediction
- Confusion matrix shows correct and incorrect predictions
- ANN generally performs well for nonlinear relationships

Expected result:

- Accuracy around 80–85% on this dataset

7. Hyperparameter Tuning

Parameters tuned:

- Number of hidden layers
- Number of neurons
- Activation functions (ReLU, Sigmoid)
- Optimizer (Adam, SGD)
- Batch size
- Epochs

Impact:

- More layers → better learning but risk of overfitting
- Learning rate affects convergence
- Batch size impacts training speed

Code

```
# =====

# ANN USING KERAS/TENSORFLOW

# =====

# Step 1: Install (only if needed in Colab)

!pip install tensorflow

# Step 2: Import Libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder,
StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Step 3: Upload Dataset in Colab manually

# Then load it

df = pd.read_csv('/content/Churn_Modelling.csv')

# Step 4: Display dataset

print("First 5 rows:\n", df.head())

print("\nDataset Info:\n")

print(df.info())

# Step 5: Preprocessing

# Select features and target

X = df.iloc[:, 3:13] # Independent variables

y = df.iloc[:, 13]   # Dependent variable

# Encode Gender

le = LabelEncoder()

X['Gender'] = le.fit_transform(X['Gender'])

# One-Hot Encode Geography

X = pd.get_dummies(X, columns=['Geography'],
drop_first=True)

# Convert to numpy array

X = X.values

y = y.values

# Step 6: Split Dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Step 7: Feature Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Step 8: Build ANN Model

model = Sequential()

# Input + Hidden Layer 1

model.add(Dense(units=6, activation='relu',
input_dim=X_train.shape[1]))

# Hidden Layer 2

model.add(Dense(units=6, activation='relu'))

# Output Layer

model.add(Dense(units=1, activation='sigmoid'))

# Step 9: Compile Model

model.compile(

    optimizer='adam',

    loss='binary_crossentropy',

    metrics=['accuracy']

)

# Step 10: Train Model
```

```

history = model.fit(
    X_train,
    y_train,
    batch_size=32,
    epochs=50,
    validation_data=(X_test, y_test)
)

# Step 11: Predictions
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

# Step 12: Evaluation
print("\nAccuracy:", accuracy_score(y_test,
y_pred))

print("\nConfusion Matrix:\n",
confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n",
classification_report(y_test, y_pred))

# Step 13: Plot Accuracy Graph
plt.plot(history.history['accuracy'])

```

```

plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])

plt.show()

# Step 14: Plot Loss Graph
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])

plt.show()

plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

Output

```

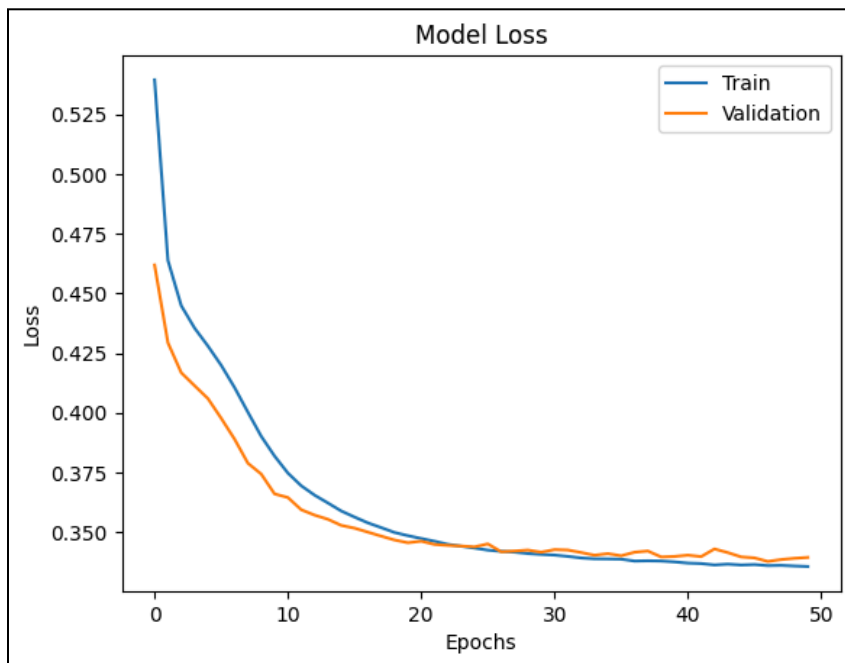
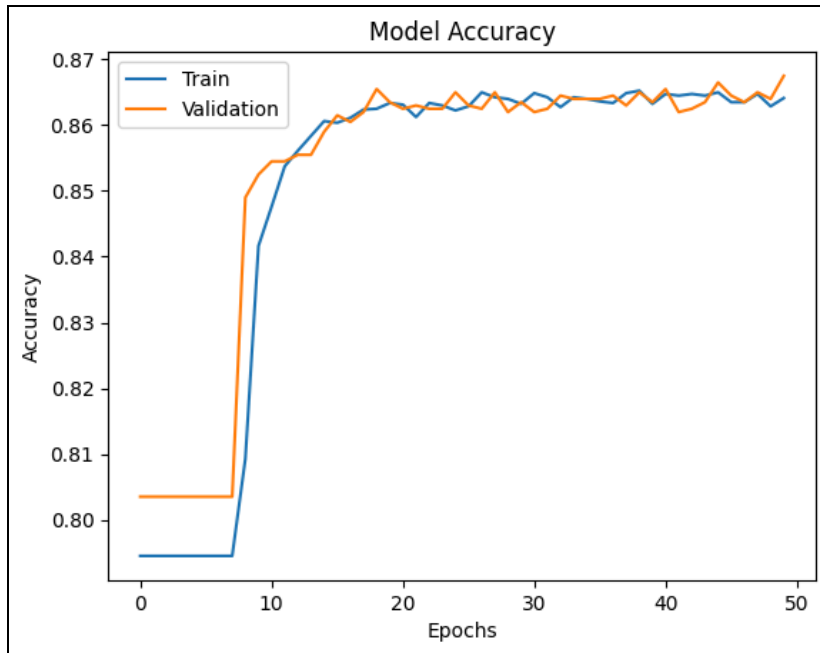
Accuracy: 0.8675

Confusion Matrix:
[[1534  73]
 [ 192 201]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.89	0.95	0.92	1607
1	0.73	0.51	0.60	393
accuracy			0.87	2000
macro avg	0.81	0.73	0.76	2000
weighted avg	0.86	0.87	0.86	2000



Conclusion

The experiment successfully implemented an Artificial Neural Network using Keras to solve a binary classification problem. The model was able to learn complex patterns in customer data and predict churn effectively. Proper preprocessing and feature scaling significantly improved model performance. Hyperparameter tuning helped in achieving better accuracy and avoiding overfitting. Overall, ANN proved to be a powerful technique for handling real-world prediction tasks.